# 08 CHAPTER
# 트리와 힙

자료구조 (파이썬) · 8주차 강의자료

※ 빈틈노트 샘플용 가상 강의자료입니다. 특정 교재·강의와 무관하며, 내용은 일반적인 자료구조 교과 내용을 바탕으로 팀이 직접 작성했습니다.

# 8장. 트리와 힙

8.1 트리란?

8.2 이진 트리

8.3 이진 트리의 순회

8.4 이진 탐색 트리

8.5 힙

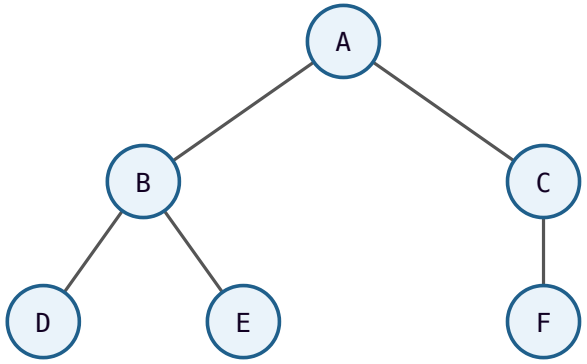
8.6 힙의 응용: 우선순위 큐와 힙 정렬

# 학습 목표

- 트리의 기본 용어(루트, 부모·자식, 단말 노드, 높이·깊이)를 설명할 수 있다
- 이진 트리의 전위·중위·후위 순회 결과를 구할 수 있다
- 이진 탐색 트리의 탐색과 삽입을 코드로 구현할 수 있다
- 힙의 삽입·삭제 과정을 그림으로 설명하고 시간 복잡도를 말할 수 있다
- 힙으로 우선순위 큐를 구현하는 이유를 설명할 수 있다

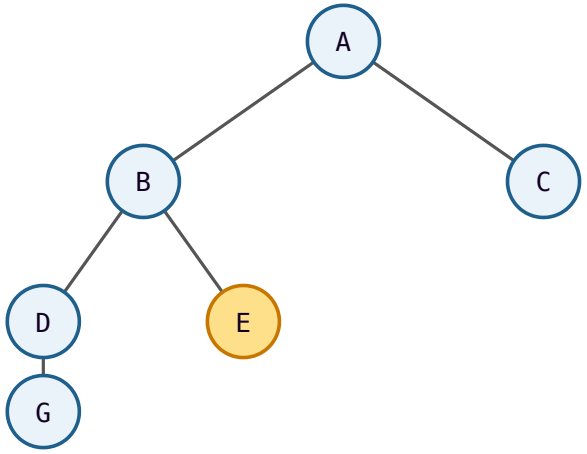
# 8.1 트리란?

- 계층적인 관계를 표현하는 비선형 자료구조
  - 예: 폴더 구조, 조직도, 가계도
- 용어
  - 루트(root): 가장 위의 노드 (A)
  - 부모·자식: 간선으로 연결된 위·아래 노드 (B는 A의 자식, D·E의 부모)
  - 단말 노드(leaf): 자식이 없는 노드 (D, E, F)
  - 서브트리: 한 노드와 그 자손 전체



# 트리의 높이와 깊이

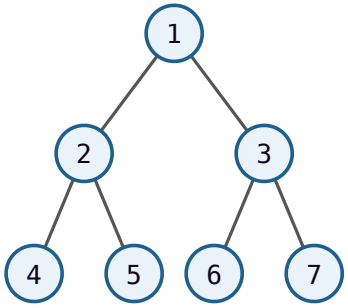
- 깊이(depth): 루트에서 그 노드까지의 경로 길이(간선 수)
  - 루트의 깊이 = 0, E의 깊이 = 2
- 높이(height): 그 노드에서 가장 먼 단말 노드까지의 경로 길이
  - 단말 노드의 높이 = 0, B의 높이 = 2, 트리의 높이 = 루트의 높이 = 3
- 깊이는 위에서 세고, 높이는 아래에서 센다



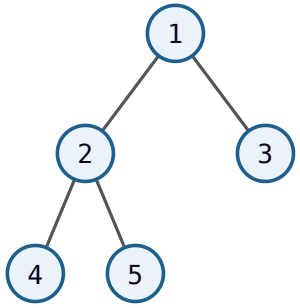

같은 노드라도 깊이와 높이는 다를 수 있다. 노드 E: 깊이 2, 높이 0

# 8.2 이진 트리

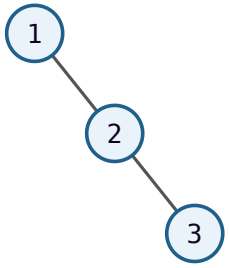
- 모든 노드가 최대 2개의 자식(왼쪽, 오른쪽)을 가지는 트리
- 종류
  - 포화 이진 트리: 모든 레벨이 꽉 참
  - 완전 이진 트리: 마지막 레벨을 제외하고 꽉 차 있고, 마지막 레벨은 왼쪽부터 채워짐 → 힙에서 사용
  - 편향 이진 트리: 한쪽으로만 자식이 있음




포화



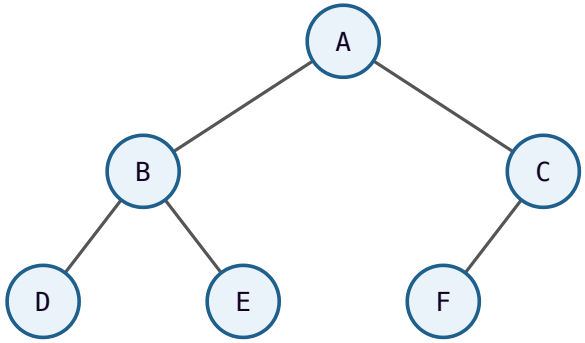

완전

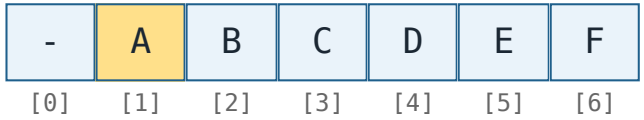



편향

# 완전 이진 트리의 배열 표현

- 완전 이진 트리는 배열에 빈칸 없이 저장할 수 있다 (루트를 인덱스 1에)
- 노드 i의 왼쪽 자식 = 2i, 오른쪽 자식 = 2i+1, 부모 = i // 2



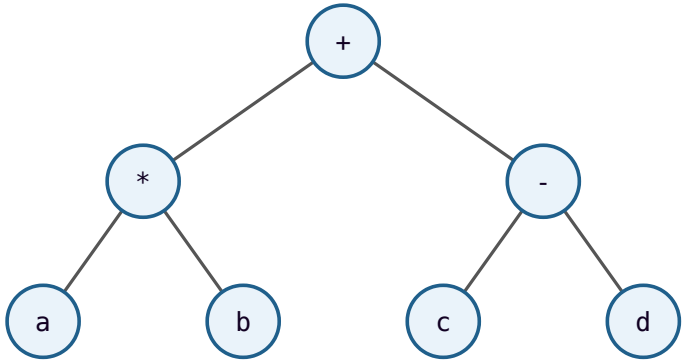

인덱스 0은 비워 둠

8.5 힙에서 이 배열 표현을 그대로 사용한다

# 8.3 이진 트리의 순회

- 순회(traversal): 트리의 모든 노드를 한 번씩 방문하는 것
- 루트(V)를 언제 방문하느냐에 따라 세 가지
  - 전위 순회(preorder): V → L → R
  - 중위 순회(inorder): L → V → R
  - 후위 순회(postorder): L → R → V
- 세 방법 모두 재귀로 간단히 구현된다
- 이진 탐색 트리를 중위 순회하면 오름차순으로 값이 나온다 (8.4)

# 순회 예제




| 순회 | 방문 순서 |
|---|---|
| 전위 | + * a b - c d |
| 중위 | a * b + c - d |
| 후위 | a b * c d - + |

수식 트리를 후위 순회하면 후위 표기식이 된다

예제: 아래 트리의 중위 순회 결과를 직접 써 보자 → a * b + c - d

# 순회 코드

```
class TNode:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right

def preorder(n):            # V L R
    if n is not None:
        print(n.data, end=' ')
        preorder(n.left)
        preorder(n.right)

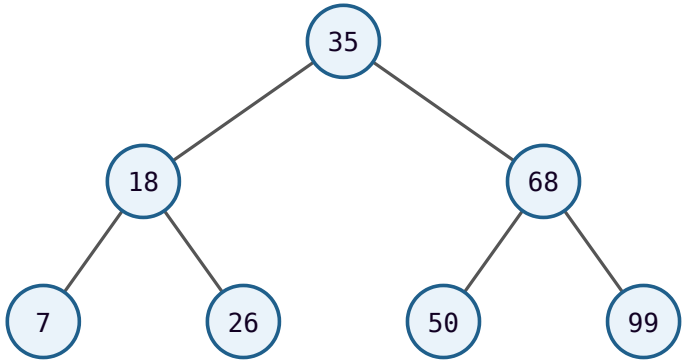
def inorder(n):             # L V R
    if n is not None:
        inorder(n.left)
        print(n.data, end=' ')
        inorder(n.right)

def postorder(n):           # L R V
    if n is not None:
        postorder(n.left)
        postorder(n.right)
        print(n.data, end=' ')
```

시간 복잡도: 노드 n개를 각각 한 번씩 방문 → O(n)

# 8.4 이진 탐색 트리

- 이진 탐색 트리(BST): 탐색을 위해 정렬 규칙을 가진 이진 트리
- 모든 노드에 대해 왼쪽 서브트리 < 노드 < 오른쪽 서브트리
- 중복된 키는 없다고 가정
- 중위 순회 결과: 7 18 26 35 50 68 99 → 오름차순



# 이진 탐색 트리의 탐색과 삽입

- 탐색: 루트부터 시작, 키가 작으면 왼쪽, 크면 오른쪽으로 내려간다
- 삽입: 탐색이 실패한 위치(None)에 새 노드를 붙인다 → 항상 단말 노드로 삽입
- 시간 복잡도: 트리의 높이 h에 비례 → O(h)
  - 균형이 잡히면 O(log n), 편향되면 O(n)

```
def search_bst(n, key):
    if n is None: return None
    if key == n.data: return n
    elif key < n.data: return search_bst(n.left, key)
    else: return search_bst(n.right, key)

def insert_bst(r, n):          # r: 루트, n: 새 노드
    if r is None: return n
    if n.data < r.data: r.left = insert_bst(r.left, n)
    elif n.data > r.data: r.right = insert_bst(r.right, n)
    return r
```

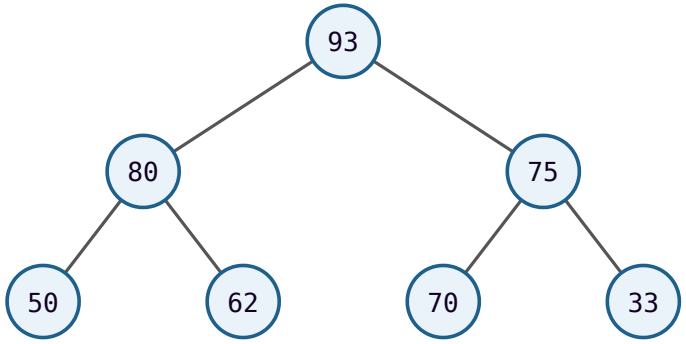
예제: 빈 트리에 35, 18, 68, 7, 26 순서로 삽입하면 앞 슬라이드의 트리가 된다

# 이진 탐색 트리의 삭제

- 삭제할 노드의 자식 수에 따라 세 가지 경우로 나뉜다
  - 경우 1 단말 노드: 그냥 제거
  - 경우 2 자식이 하나: 그 자식을 부모에 연결
  - 경우 3 자식이 둘: 오른쪽 서브트리의 최솟값(후계자)을 찾아 그 값을 복사하고, 후계자 노드를 삭제
- 경우 3이 가장 복잡하다. 후계자는 자식이 최대 하나이므로 경우 1·2로 환원된다
- 자세한 코드는 다음 주 (교재 8.4절 코드 8.5)

# 8.5 힙이란?

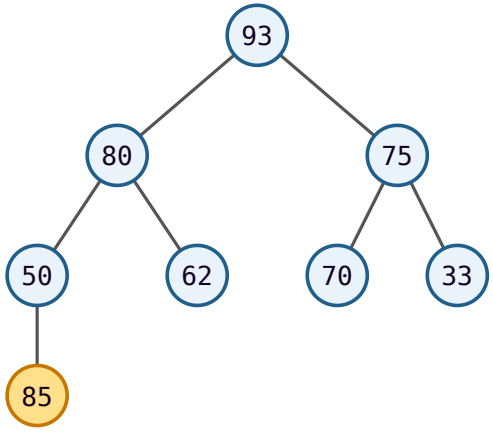
- 힙(heap): 완전 이진 트리이면서, 모든 노드가 힙 조건을 만족하는 트리
  - 최대 힙: 부모 ≥ 자식 → 루트가 최댓값
  - 최소 힙: 부모 ≤ 자식 → 루트가 최솟값
- 형제 사이의 순서는 없다 (정렬된 것이 아님)
- 완전 이진 트리이므로 배열로 표현한다 (부모 i → 자식 2i, 2i+1)



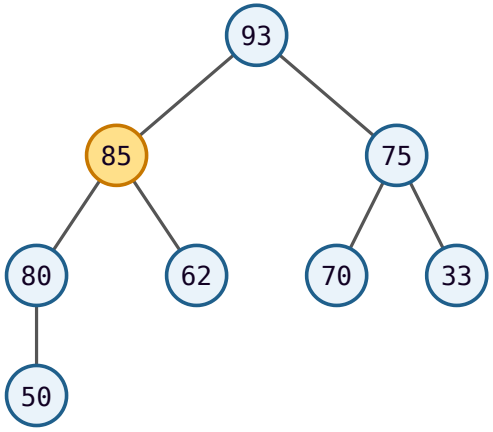

| - | 93 | 80 | 75 | 50 | 62 | 70 | 33 |
|---|---|---|---|---|---|---|---|
| [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] |

# 힙의 삽입 연산

- ① 새 항목을 마지막 위치(배열의 끝)에 넣는다 → 완전 이진 트리 유지
- ② 부모와 비교해 더 크면 부모와 교환하며 위로 올라간다 (업힙, sift-up) → 힙 조건 회복
- 시간 복잡도: 트리 높이만큼만 올라가므로 O(log n)



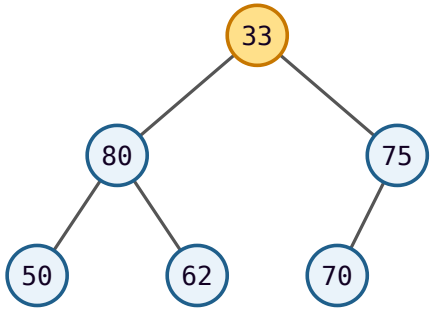

① 85를 마지막에 추가



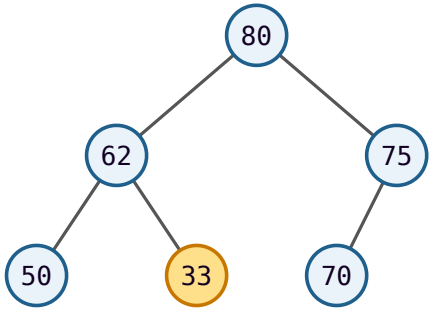

② 50, 80과 교환하며 올라감 → 93보다 작으므로 정지

# 힙의 삭제 연산

- 힙에서 삭제는 항상 루트(최댓값)를 꺼내는 것
- ① 루트를 꺼내고, 마지막 노드를 루트 자리로 옮긴다
- ② 두 자식 중 더 큰 자식과 비교해 작으면 교환하며 내려간다 (다운힙, sift-down)
- 시간 복잡도: O(log n)




① 93 삭제 후 33을 루트로




② 80, 62와 교환하며 내려감 → 결과 [80, 62, 75, 50, 33, 70]

# 최대 힙 코드 (배열 기반)

```
class MaxHeap:
    def __init__(self):
        self.heap = [0]            # 인덱스 0은 사용하지 않음
    def size(self): return len(self.heap) - 1

    def insert(self, n):           # 업힙
        self.heap.append(n)
        i = self.size()
        while i != 1 and n > self.heap[i // 2]:
            self.heap[i] = self.heap[i // 2]
            i = i // 2
        self.heap[i] = n

    def delete(self):              # 다운힙
        root = self.heap[1]
        last = self.heap.pop()
        if self.size() == 0: return root
        parent, child = 1, 2
        while child <= self.size():
            if child < self.size() and self.heap[child] < self.heap[child + 1]:
                child += 1         # 더 큰 자식 선택
            if last >= self.heap[child]: break
            self.heap[parent] = self.heap[child]
            parent, child = child, child * 2
        self.heap[parent] = last
        return root
```

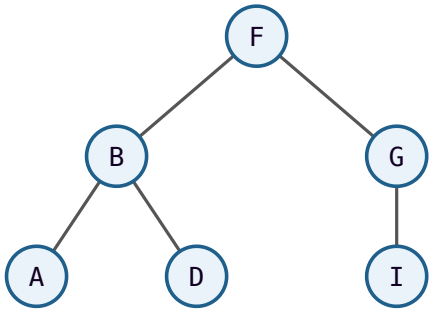
# 8.6 힙의 응용: 우선순위 큐와 힙 정렬

- 우선순위 큐: 들어온 순서가 아니라 우선순위가 높은 항목부터 나가는 큐
  - 연산: insert(item), remove() → 가장 높은 우선순위 반환
  - 힙으로 구현하면 두 연산 모두 O(log n) (배열·연결 리스트는 둘 중 하나가 O(n))
  - 응용: 운영체제 작업 스케줄링, 다익스트라 최단 경로, 허프만 코딩
- 힙 정렬: n개를 모두 힙에 넣고(O(n log n)) 하나씩 꺼내면(O(n log n)) 정렬됨
  - 전체 O(n log n), 추가 메모리 거의 없음 (제자리 정렬 가능)

| 구현 | 삽입 | 삭제(최댓값) |
|---|---|---|
| 정렬 안 된 배열 | O(1) | O(n) |
| 정렬된 배열 | O(n) | O(1) |
| 힙 | O(log n) | O(log n) |

# 연습문제

1. 오른쪽 트리의 전위·중위·후위 순회 결과를 각각 쓰시오
2. 오른쪽 트리에서 노드 D의 깊이와 높이, 트리 전체의 높이를 구하시오
3. 빈 이진 탐색 트리에 50, 30, 70, 20, 40, 60을 순서대로 삽입한 결과를 그리시오
4. 최대 힙 [ -, 90, 70, 80, 30, 50, 60 ]에 95를 삽입하는 과정을 단계별로 그리고, 교환 횟수를 쓰시오
5. 삽입 전의 힙 [ -, 90, 70, 80, 30, 50, 60 ]에서 최댓값을 삭제한 뒤의 배열을 쓰시오
6. 우선순위 큐를 힙으로 구현하면 정렬된 배열보다 유리한 이유를 시간 복잡도로 설명하시오



# 요약

- 트리 용어: 루트, 부모·자식, 단말, 깊이(위에서)와 높이(아래에서)
- 이진 트리 순회: 전위 VLR, 중위 LVR, 후위 LRV → 재귀로 구현, O(n)
- 이진 탐색 트리: 왼쪽 < 루트 < 오른쪽, 탐색·삽입 O(h)
- 힙: 완전 이진 트리 + 힙 조건, 배열로 표현
- 힙 삽입 = 마지막에 넣고 업힙, 힙 삭제 = 루트 꺼내고 마지막을 올려 다운힙, 둘 다 O(log n)
- 우선순위 큐는 힙으로, 힙 정렬은 O(n log n)

수고하셨습니다 · 다음 주: 이진 탐색 트리의 삭제와 AVL 트리